Secure Smartphone-Based Vehicle Access Control: A Hardware-Backed Asymmetric Signing Approach for Fleet Management

CONTENTS

ABSTRACT

This paper presents the design and implementation of a smartphone-based digital car key system for fleet and rental vehicle management. The prototype replaces physical keys with cryptographically signed Bluetooth Low Energy (BLE) commands, verified by a Python based Vehicle Access Control Unit (VACU) and sent to a vehicle test rig through real Controller Area Network (CAN) messages.

An ECDSA P-256 signing scheme is used, with the private key being generated and securely stored in the Android Keystore on a per-user basis. The corresponding public key is only stored in the cloud and is used by the VACU to verify commands. This removes the need to store sensitive signing data outside the user's device.

Replay attacks are prevented using a combination of timestamps and a short-term nonce cache, which blocks both delayed and immediate replay attempts. A threat model is demonstrated with seven different attack scenarios, all of which are successfully detected, with alerts updating on a cloud-based monitoring dashboard.

INTRODUCTION

The automotive industry is moving toward digital key systems that allow smartphones to replace traditional car keys. Standards such as the Car Connectivity Consortium (CCC) Digital Key specification cover the hardware side of this, but fleet and rental operators have additional challenges. They need to assign, revoke, and monitor keys remotely across many vehicles, often without being able to rely on specialised hardware on the user's device.

One of the main design questions is where the signing key lives and how the vehicle verifies a command to be valid. One option is to keep a shared secret in the cloud database, but this means the same secret must exist on the phone, in the cloud and on the vehicle side. Compromise of any one of these will result in the key being exposed.

This project takes an asymmetric approach instead, with each user's device generates its own key pair using the Android Keystore. The private key never leaves the device and only the public key is uploaded to the cloud for the vehicle to verify against. This means the user's device is the only part of the system that can create a valid command.

SYSTEM ARCHITECTURE

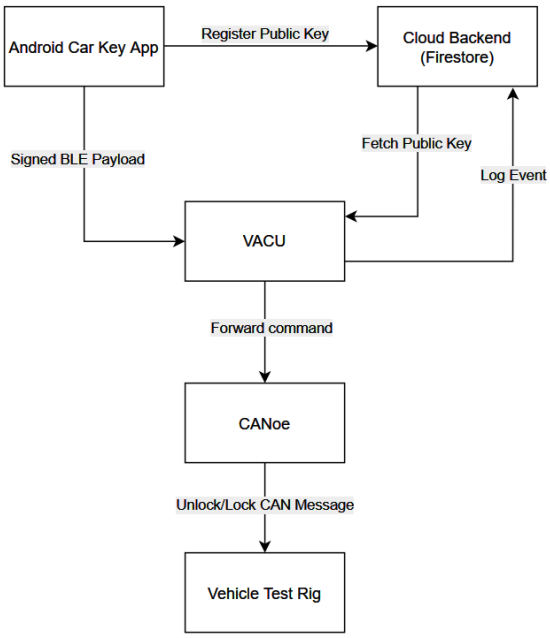

Figure 1: System architecture. Signed commands travel over BLE from phone to VACU; the VACU fetches the user's public key from the cloud to verify.

The prototype consists of four components operating end-to-end.

Android Car Key App: Built with Kotlin and Jetpack Compose following the MVVM architectural pattern. The app authenticates users via Firebase Authentication with biometric verification (fingerprint or face) used on session resume, preventing use of a stolen unlocked device. On first sign in, the app generates an ECDSA P-256 key pair in the Android Keystore under a per-user alias and registers the encoded public key to the user's profile in Firestore. All subsequent payload signing is performed by the Keystore using this hardware-based private key.

Python VACU: This component runs on a PC and implements the BLE GATT central role, using the bleak library. The VACU receives signed payloads and checks them against a four stage validation pipeline before sending any command. On validation success, commands are sent to Vector CANoe through a COM interface, which generates and transmits CAN messages to the vehicle test rig. The prototype demonstrates the rig turning on and off with unlock and lock commands, through simulating the vehicle’s real ignition CAN messages.

Firebase Cloud Backend: Firestore stores all user profiles, public keys, vehicle records, provisioned digital keys, and an event audit log. The Firestore Security Rules enforce role-based access - drivers can only read their own key document and may update only the publicKey field on their own user record, managers cannot impersonate users, and unauthenticated access is blocked entirely. The VACU accesses Firestore via the Firebase Admin SDK through using a service account, which bypasses client side rules and can’t be impersonated by compromised user session.

Manager App and Dashboard: The manager interface is accessed by logging into the Android app with the manager role. It allows fleet managers to provision and revoke digital keys, manage users and vehicles, and view access logs. A monitoring dashboard is implemented using Streamlit, and provides a view of all lock, unlock, and failure events across the fleet in real time. It also shows a security overview of all detected attack events in the system, with their corresponding attack type.

SECURITY DESIGN

Payload Format and Signing

Every BLE command follows the format:

[command]:[nonce]:[timestamp_ms]:[signature]

The signature is generated by the Android Keystore using ECDSA-SHA256 over the first three fields. The message is encoded using Base64 for transport. The nonce is a randomly generated 32-character hex string, with the timestamp using a Unix millisecond value. The payload is deemed valid only if its timestamp falls within a 30 second age window with a 10-second clock skew tolerance, which gives an effective validity range of 40 seconds.

Hardware-Backed Asymmetric Signing

When a user signs in to the app for the first time, an ECDSA P-256 key pair is generated using Android Keystore. The key is stored under a unique alias based on the user ID. The private key is stored inside Android Keystore and cannot be extracted.

The public key is encoded and stored in the user's Firestore record. This lets the VACU verify any command sent by that user without needing access to the user's private key.

When a command is received, the VACU retrieves the user's public key and uses it to verify the signature. Since the private key never leaves the device, neither the cloud backend nor the VACU can generate valid commands.

This approach improves security compared to shared secret methods, as there is no single piece of sensitive data stored in multiple locations. However, it introduces a dependency on the cloud to retrieve the public key when needed. To reduce this impact, the VACU caches public keys for a short period (30 seconds), balancing performance with the ability to revoke keys quickly.

Four-Stage VACU Validation Pipeline

Each received command is checked in four stages before any action is taken:

Stage 1: Format, command, and timestamp validation:

The system checks that the message is correctly structured, that the command is valid, and that the timestamp is within the allowed time window. Messages that are too old or set in the future are rejected.

Stage 2: Nonce uniqueness:

A temporary in-memory cache stores recently seen nonces. If a nonce is reused, the message is rejected, preventing replay attacks within the valid time window.

Stage 3: Key status check:

The VACU checks the cloud database to confirm that the key is still active for the vehicle. Revoked or expired keys are rejected. A short cache is used to reduce database load.

Stage 4: Signature verification:

The system verifies the ECDSA signature using the user's public key. If the signature does not match, the command is rejected.

THREAT EVALUATION

Seven attack scenarios were simulated against the live VACU validation pipeline using a dedicated threat simulation module. All rejections were logged to Firestore in real time and verified on the monitoring dashboard.

Attack	Mechanism	Rejection Stage
Replay - expired timestamp	Valid payload retransmitted after 40-second window	Stage 1 - timestamp check
Replay - within window	Valid payload retransmitted immediately using same nonce	Stage 2 - nonce uniqueness
Payload tampering	Command field changed from lock to unlock	Stage 4 - signature mismatch
Malformed payload	Missing fields, empty payload	Stage 1 - format validation
Unknown command injection	Unsupported command (e.g. start_engine)	Stage 1 - command validation
Future timestamp	Timestamp set 1 hour ahead	Stage 1 - clock skew rejection
Revoked key	Key revoked in Manager App during active session	Stage 3 - no active key found

CONCLUSION AND FUTURE WORK

This work shows that using hardware-backed asymmetric cryptography is a practical and secure approach for smartphone-based vehicle access control. By keeping private keys on the user's device and only storing public keys in the cloud, the system avoids exposing sensitive signing material while still allowing flexible key management.

Future work includes enabling BLE pairing and bonding so that the BLE traffic itself is encrypted, rather than relying on the payload signature alone for integrity. A push notification channel from the cloud to the VACU would remove the small delay between a manager revoking a key and the VACU stopping accepting it. Adding a check that confirms the VACU has not been tampered with before it accepts any commands would extend protection past the current software-only setup. Allowing a user to move their digital key from one phone to another would also be needed for production use, since the private key is currently tied to the device that generated it - the CCC Digital Key 3.0 specification defines a process for this. Finally, replacing ECDSA with a signature scheme designed to resist quantum attacks is worth investigating for long-term deployments.

Task	Prompt
Android Keystore ECDSA P-256 setup	Help me setup ECDSA P-256 key pair using Android Keystore. Private key is stored per user
Migrating from HMAC signing to asymmetric	Whats the best way to migrate using per key HMAC secrets in the cloud to asymmetric signing with android keystore
Python ECDSA signature verification	Have an X.509 public key encoded as base64, how do I verify ecdsa signature in python
Firestore quota limits	Im hitting the firestore quota when im running the demo, help me troubleshoot
Firestore Security Rules	Help write security rules for firestore for role based access for drivers and managers, drivers can only change their own public key
Unit tests	Look at my payload format and suggest some unit tests to verify structure
Streamlit dashboard time range	Help add a time selector for my security graph on Streamlit, for 1 day, 3 day, week, month,
Plotly barchart fix	Chart is sorting security events alphabetically rather than by date, help me fix
BLE duplicate UUID issue	Getting a duplicate UUID error from VACU when starting the BLE on Android
BLE reconnection	How would I make the BLE gatt loop reconnect automatically on connection loss?